



DEEP LEARNING AND NATURAL LANGUAGE PROCESSING

LECTURE 10: RETRIEVAL-AUGMENTED GENERATION AND VECTOR DATABASES

Dr. Adewale Akinfaderin

SEAS 8520-D6A · Summer 2026



Table of Contents

1 The Knowledge Problem

- ▶ The Knowledge Problem
- ▶ Embeddings and Similarity Search
- ▶ Vector Databases
- ▶ The RAG Pipeline
- ▶ Advanced RAG
- ▶ Evaluating RAG
- ▶ Case Study: RAG on One Laptop

Where we left off. The previous lecture covered every way to change a model's *behavior*: decoding control, prompting, LoRA and QLoRA fine-tuning, and preference optimization. Today we solve the complementary problem: **how do you give a frozen model access to knowledge it was never trained on?**

1. **The knowledge problem:** what weight updates cannot fix.
2. **Embeddings and similarity search:** dense retrieval, BM25, and when each wins.
3. **Vector databases:** exact search, ANN indexes, IVF and HNSW.
4. **The RAG pipeline:** ingestion, chunking, retrieval, synthesis.
5. **Advanced RAG:** query transformation, hybrid search, reranking.
6. **Evaluation:** ranking metrics, the RAG triad, LLM judges.
7. **Case study:** seven retrieval strategies, measured on one laptop.

Fine-tuning teaches *how to respond*. Three knowledge problems survive any amount of it:

- **Staleness.** Weights encode the world as of the training cutoff. Yesterday's policy change, this morning's ticket, and next quarter's org chart are all invisible, and retraining for every update is economically absurd.
- **Privacy.** Your contracts, medical records, and internal wikis were never in the pretraining corpus. Fine-tuning on them bakes private data into weights that may be served to every user.
- **Attribution.** A parametric answer arrives with no citation. Regulated settings (healthcare, legal, finance) need to point at the exact source page.

The architectural answer

Separate the knowledge from the model: keep documents in an external store, retrieve what is relevant at query time, and let the frozen LLM read it. Updating knowledge becomes a database write instead of a training run.



Table of Contents

2 Embeddings and Similarity Search

- ▶ The Knowledge Problem
- ▶ Embeddings and Similarity Search
- ▶ Vector Databases
- ▶ The RAG Pipeline
- ▶ Advanced RAG
- ▶ Evaluating RAG
- ▶ Case Study: RAG on One Laptop

The operational constraint. A corpus of 10,000 documents holds on the order of 10^7 tokens. Even models with very long context windows cannot take it all:

- Attention cost and prompt cost grow with input length; paying for 10^7 tokens per query is ruinous.
- Models attend unevenly across very long inputs, so relevant evidence buried mid-context is often ignored (Liu et al., 2023).

So every query needs a **selection step**: from millions of candidate passages, pick the handful worth reading.

Information retrieval supplies that step. Two families of methods, both producing a ranked list:

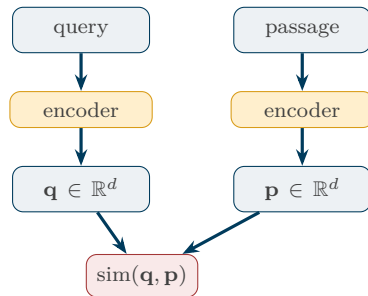
- **Sparse (lexical)**: match the query's exact terms. Decades of search-engine engineering.
- **Dense (semantic)**: match meaning through learned vector representations.

The next frames build both, because production systems end up needing both.

An **embedding model** maps a passage of text to a single vector $\mathbf{v} \in \mathbb{R}^d$ (typically $d = 384$ to 4096) such that semantically similar texts land close together.

- Earlier lectures built word embeddings and contextual token representations; retrieval embeddings pool an entire passage into one vector.
- Trained **contrastively**: pull (query, relevant passage) pairs together, push random passages apart, over hundreds of millions of pairs.
- Sentence-BERT (Reimers and Gurevych, 2019) established the recipe most modern embedders still follow.

The **bi-encoder pattern**:



Each side is encoded **independently**, so all passage vectors can be computed offline and stored. Query time costs one encoder pass plus a vector search.

Cosine similarity compares direction and ignores magnitude:

$$\cos(\mathbf{q}, \mathbf{p}) = \frac{\mathbf{q} \cdot \mathbf{p}}{\|\mathbf{q}\| \|\mathbf{p}\|} \in [-1, 1]$$

Dot product rewards magnitude as well:

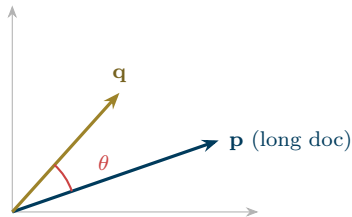
$$\mathbf{q} \cdot \mathbf{p} = \sum_i q_i p_i.$$

Euclidean distance measures separation:

$$\|\mathbf{q} - \mathbf{p}\|_2.$$

The identity everyone exploits

After normalizing every vector to unit length, all three orderings agree, and cosine similarity *is* the dot product. Retrieval over a whole corpus becomes one matrix-vector multiply $E\mathbf{q}$.



Cosine keeps a short query and a long paragraph comparable: only the angle θ matters, so verbose documents earn no automatic advantage.

- **Sparse representation:** a document is a vector with one dimension per vocabulary word (tens of thousands of dimensions), almost all zero. Weights come from term frequency and rarity, the TF-IDF family from earlier in this course.
- **Inverted index:** for each term, store the list of documents containing it. A query touches only the posting lists of its own terms, which is why lexical engines answer in milliseconds over billions of documents.
- **Inverse document frequency** (Spärck Jones, 1972) supplies the rarity weight:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

where N is the corpus size and $n(q_i)$ counts documents containing term q_i . A term that appears everywhere (*the*, *patient*) carries almost no signal; a rare term (*bioluminescence*) carries a lot.

Karen Spärck Jones's IDF insight from 1972 still sits, essentially unchanged, inside every production search engine.

BM25 (Robertson et al., 1995) scores document D against query $Q = \{q_1, \dots, q_n\}$:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) (k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{\text{avgdl}}\right)}$$

- $f(q_i, D)$: how often term q_i occurs in D ; more occurrences mean more evidence, with diminishing returns.
- $k_1 \approx 1.2$: controls saturation. The term factor rises toward the ceiling $k_1 + 1$ and stops, so the 10th occurrence of *infection* barely outscores the 3rd, and keyword stuffing cannot win.
- $b \approx 0.75$: length normalization; $|D|/\text{avgdl}$ penalizes documents long relative to the corpus average, since long documents match everything by accident.
- $\text{IDF}(q_i)$: rare query terms dominate the score.

BM25 needs only three tuned constants and no training data or GPU, which is why it remains the baseline that dense retrievers are measured against, and it still wins on exact strings.

Vocabulary mismatch. A user asks about “*connecting wireless headphones*”; the manual says “*pairing a Bluetooth audio device.*” No shared content words, so BM25 scores near zero while a dense embedder places the two side by side.

Exact identifiers. For error code E-4013 or form OSHA-300A, embeddings blur rare strings into their neighborhood and return something *related*; BM25 returns the literal match. Whole fields live on this: drug names (*imatinib*, CHEMBL941), gene symbols (*BRCA1*), legal citations (*42 U.S.C. § 1983*), where a near-synonym is the wrong molecule or statute.

Query type	Dense (embeddings)	Sparse (BM25)
Paraphrased, conceptual	strong	weak
Exact codes, IDs, names	weak	strong
Domain jargon unseen in training	unreliable	strong if term matches
Cross-lingual	possible	no
Index update cost	re-embed	cheap

Since real query streams mix both types, production systems run both and merge the results. Hybrid search returns in the Advanced RAG section.



Table of Contents

3 Vector Databases

- ▶ The Knowledge Problem
- ▶ Embeddings and Similarity Search
- ▶ **Vector Databases**
- ▶ The RAG Pipeline
- ▶ Advanced RAG
- ▶ Evaluating RAG
- ▶ Case Study: RAG on One Laptop

Definition: Vector database

A data store specialized for high-dimensional vectors, providing three coupled capabilities: **storage** of vectors with their payloads and metadata, **approximate nearest-neighbor indexing** for fast similarity search, and **filtered queries** that combine vector similarity with Boolean predicates.

- The interface is small: `upsert(id, vector, metadata)` and `query(vector, k, filter)` cover most usage.
- The engineering lives underneath: keeping the index fresh under inserts and deletes, sharding across machines, and making filters compose with the ANN index.
- The stored vector is only a key. The payload (chunk text, source document, page number) is what the RAG pipeline actually consumes.

	Relational DB	Vector DB
Unit of data	row with typed columns	vector + payload + metadata
Core query	exact predicate match	nearest neighbors of a vector
Answer semantics	exact, deterministic	approximate, ranked by similarity
Index structure	B-tree, hash	IVF, HNSW, PQ
“Equality”	<code>WHERE id = 42</code>	$\cos(\mathbf{q}, \mathbf{p}) > 0.8$
Typical failure	missing row	near miss ranked too low

- The two are complements: production RAG systems routinely keep documents in an object store, metadata in Postgres, and vectors in a vector index.
- The convergence is real, though: Postgres with `pgvector` adds ANN search to a relational engine, and most vector DBs added metadata filtering. The categories are merging from both sides.

- Exact k -NN compares the query against **every** stored vector: $O(N \cdot d)$ per query. For $N = 10^8$ vectors at $d = 768$, that is $\sim 7.7 \times 10^{10}$ multiply-adds *per query*, before ranking.
- Classic space-partitioning trees (k -d trees) collapse in high dimensions: with d in the hundreds, their pruning stops working and they degrade to a full scan.
- **The curse of dimensionality:** as d grows, distances between random points concentrate; the nearest and farthest neighbor differ by less and less, so geometric shortcuts that worked in 2-D stop discriminating.

The engineering escape

Give up the guarantee. Accept the *probably* nearest neighbors in exchange for query times that are sublinear in N . Every production vector index makes this trade.

Every ANN index exposes a single recall-versus-speed dial:

- Turn toward **recall**: probe more cells, search wider, answer slower, approaching the exact result.
- Turn toward **speed**: search less of the index; risk missing the true neighbor and returning its runner-up instead.

For RAG this is usually a good trade: the second-best passage about a policy tends to answer the question too, so a small recall loss rarely changes the generated answer while cutting latency by orders of magnitude.

Two ways to realize the trade-off

The dominant index families take different routes: **IVF** partitions the space and scans a few cells; **HNSW** walks a layered proximity graph. Both next.

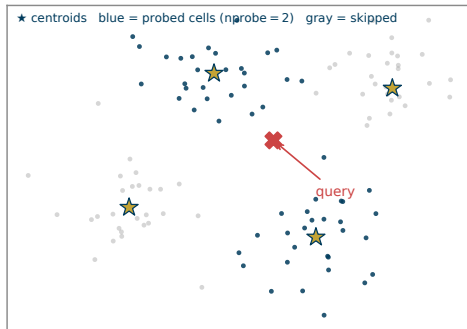
Inverted File Index (IVF):

1. **Index time:** run k -means to split the vector space into n_{list} cells (a common heuristic: $n_{\text{list}} \approx \sqrt{N}$). Each vector is assigned to its nearest centroid's list.
2. **Query time:** compare the query with all centroids, pick the n_{probe} nearest cells, and scan only those lists.

Cost drops from N comparisons to roughly $n_{\text{list}} + n_{\text{probe}} \cdot \frac{N}{n_{\text{list}}}$.

Failure mode: a true neighbor sitting just across a cell boundary is invisible unless n_{probe} is raised, which is exactly the recall dial.

IVF searches only the cells nearest the query

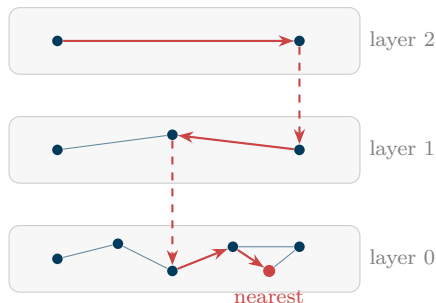


Hierarchical Navigable Small World

(Malkov and Yashunin, 2018) stacks proximity graphs:

- **Bottom:** all vectors, each linked to its M nearest neighbors (M from 16 to 64).
- **Upper layers:** sparse long-range links, like a skip list.
- **Search:** greedy descent from the top, $O(\log N)$; **efSearch** is the recall dial.

Managed-service default: high recall at millisecond latency, no training step, incremental inserts. The cost is RAM, since the graph lives in memory beside the vectors.



Red path: search enters at the top, rides long links, then descends (dashed) and refines locally.

The previous lecture shrank *model weights* to 4 bits; the same pressure applies to stored vectors. 10^8 vectors at $d = 768$ in FP32 is ~ 300 GB of RAM before any graph overhead.

- **Product quantization** (Jégou et al., 2011): split each vector into m subvectors (e.g., 96 chunks of 8 dims), run k -means with 256 centroids *within each chunk*, and store only the centroid IDs.
- Storage per vector: $768 \times 4 \text{ B} = 3,072 \text{ B} \rightarrow 96 \times 1 \text{ B} = 96 \text{ B}$, a $32\times$ reduction.
- Distances are computed against centroids via lookup tables, so search speeds up as well; recall drops slightly, and a reranking pass over exact vectors recovers most of it.
- Composability: IVF+PQ (partition, then compress within cells) is the standard configuration in FAISS for billion-scale corpora.

Real queries carry predicates: “...*in contracts signed after 2024, department = oncology.*”

- **Post-filtering:** run ANN search first, then discard non-matching results. Simple, but a selective filter can leave zero survivors out of the top k , so the system must re-query with larger k .
- **Pre-filtering:** restrict the search to matching vectors from the start. Correct results, but it fights the index: an HNSW graph loses its connectivity guarantees when most nodes are masked out, so engines fall back to brute force below a selectivity threshold.
- Production engines pick a plan per query based on estimated filter selectivity, exactly like a relational query optimizer choosing between an index scan and a table scan.

Design consequence

Decide the filterable fields *before* ingestion. Metadata attached to each chunk (source, date, department, access level) is what makes vector search deployable inside an organization with permissions.



The Vector Search Ecosystem

3 Vector Databases

System	Strengths	Trade-offs
FAISS	Reference IVF/HNSW/PQ, GPU, fastest raw search	Library only: no persistence, filters, or serving
pgvector	Rows and vectors in one Postgres; SQL filters, transactions	Manual tuning; strains past tens of millions
OpenSearch	BM25 and HNSW in one engine; native hybrid, mature ops	Heavy to run, memory-hungry
Qdrant	Fast Rust engine, strong meta-data filtering	Younger ecosystem
Milvus	Scales to billions, many index types	Operationally complex; wants a cluster
Weaviate	Built-in hybrid and embedding modules	Opinionated, resource-heavy
Chroma	Simple for local and dev use	Not for large-scale production
Pinecone	Zero-ops managed, serverless autoscaling	Proprietary, per-vector cost, data leaves your VPC
S3 Vectors	Object-storage economics for huge cold corpora	Higher latency, newer and feature-limited

Under $\sim 10^5$ vectors, exact search in NumPy beats any index for simplicity, which is what the case study at the end does. Above that, filters and multi-tenancy usually decide the choice before raw speed does; if the data already lives in Postgres, **pgvector** removes a system from the architecture



Table of Contents

4 The RAG Pipeline

- ▶ The Knowledge Problem
- ▶ Embeddings and Similarity Search
- ▶ Vector Databases
- ▶ The RAG Pipeline
- ▶ Advanced RAG
- ▶ Evaluating RAG
- ▶ Case Study: RAG on One Laptop

Definition: RAG

An architecture that answers queries by **retrieving** relevant passages from an external corpus and **generating** a response conditioned on them. Knowledge lives in a searchable store; the language model supplies reading comprehension and synthesis.

- Lewis et al. (2020) coined the term, framing it as combining **parametric memory** (the model's weights) with **non-parametric memory** (a retrievable index).
- The separation is the point: facts change by updating the index, behavior changes by changing the model, and the two upgrade independently.
- Grounding also changes failure behavior: an answer built from retrieved text can cite its sources, and a retrieval miss is detectable in a way that a hallucination is not.

OFFLINE: ingestion



ONLINE: per query



- The **same embedding model** must serve both rows; mixing embedders puts queries and chunks in incompatible spaces.
- The offline row runs continuously as documents change; the online row runs per request with a latency budget.

- **Parsing is unglamorous and decisive.** PDFs interleave headers, footers, footnotes, and multi-column layouts; naive text extraction shreds them. A financial table split by character count detaches numbers from their row labels, and retrieval faithfully serves the fragments.
- **Tables and figures** need dedicated handling: extract tables to structured form (or markdown), caption figures, and keep section headers attached to their body text.
- **Metadata extraction** happens here too: source, title, date, section, access level. Whatever is discarded during ingestion becomes unfilterable and uncitable forever after.

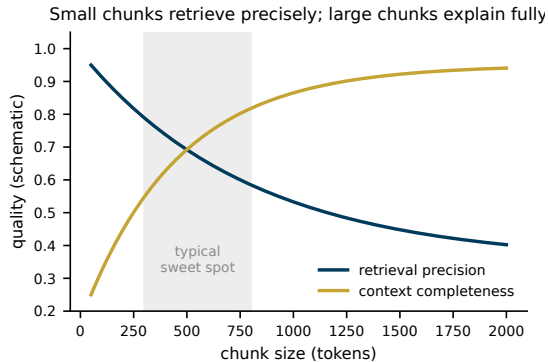
The uncomfortable rule

Retrieval quality has a hard ceiling set at ingestion. A better embedding model cannot recover information that parsing destroyed.

- Documents must be split before embedding, for two separate reasons:
 - **Model limits:** embedding models truncate input (commonly at 512 or 8192 tokens); a 40-page policy cannot become one vector without losing most of itself.
 - **Retrieval resolution:** one vector per document forces the query to match the document's *average* topic. A page about ATP surface testing inside an infection-control manual is invisible if the whole manual is one point in space.
- The chunk is therefore the **unit of retrieval**, the unit of relevance, and the unit the LLM eventually reads. Every downstream metric is computed over chunks.
- Chunk boundaries also set the **context the model receives**: a chunk that ends mid-sentence hands the generator a cliffhanger.

Strategy	Mechanism	Trade-off
Fixed-size + overlap	every n tokens, adjacent chunks share an overlap	simple baseline; cuts mid-thought
Sentence / semantic	split where embedding similarity between consecutive passages drops	coherent chunks; unpredictable sizes
Document-aware	split on headers, sections, list and table boundaries	best for structured docs (policies, manuals, APIs)
Hierarchical (small-to-big)	embed small child chunks, return the larger parent section	precise matching + rich context; two-level book-keeping
Late chunking	encode the full document first, derive chunk vectors from token states	cross-chunk context preserved; costlier encoding

Most teams start with fixed-size + overlap, then move to document-aware once retrieval errors trace back to bad boundaries. The case study uses page-level chunks, the natural document-aware split for these policy PDFs.



- **Small chunks** give each vector one clear topic, so ranking is sharp; the winner may lack the surrounding context needed to answer.
- **Large chunks** carry full explanations; their vectors average several topics and match everything weakly.
- Overlap (10–20%) protects boundary sentences at the cost of storage and duplicate retrieval hits.
- Small-to-big retrieval gets both ends: match on the child, hand the parent to the generator.

The retrieved chunks are injected into a rigid template that constrains the model to the evidence:

```
1 def build_rag_prompt(context_chunks, question):
2     context = "\n\n---\n\n".join(context_chunks)
3     return f"""Answer the question using ONLY the context below.
4 Answer every part of the question. If the context does not
5 contain the answer, say so.
6
7 Context:
8 {context}
9
10 Question: {question}"""
```

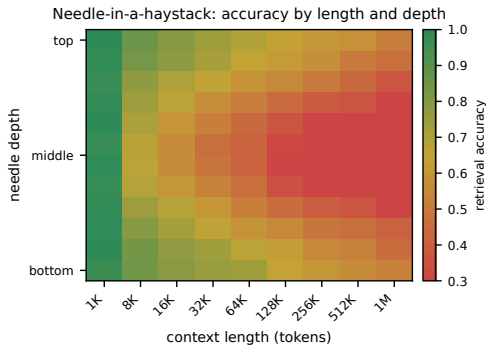
- The “say so” escape hatch matters: without it, the model answers from parametric memory whenever retrieval fails, which silently converts retrieval misses into hallucinations.
- Adding “cite the source of each claim” plus chunk identifiers in the context allows attribution at negligible cost.

Every stage has options, and a handful of decisions explain most of the variance in answer quality, latency, and cost:

1. **Chunking strategy:** sets what retrieval can see (two frames ago).
2. **Embedding model and indexing pipeline:** defines what *similar* means; inconsistent preprocessing between ingestion and query makes the right passage invisible.
3. **Retrieval method:** dense, sparse, or hybrid; hybrid usually wins in production because query streams mix paraphrases with exact identifiers.
4. **Reranking and post-processing:** a second-stage relevance model often improves quality more than any tweak to the first-stage retriever.
5. **Top- k and context budget:** too few chunks misses evidence, too many dilutes it and reintroduces the mid-context blind spot.

- **Vague queries.** Users type “*how does it work?*”; the embedded query is near nothing in particular, and top- k returns generic chunks.
- **Vocabulary and granularity mismatch.** The question spans two documents, or uses words the corpus never uses; a single fixed retrieval step has no way to adapt.
- **Position bias with many chunks.** With many chunks stuffed in, the model over-weights the first and last ones (Liu et al., 2023) and may quote an irrelevant chunk simply because it was retrieved.
- **Generation on bad retrieval.** When retrieval returns junk, the basic pipeline generates anyway. Most “*RAG hallucinations*” observed in production trace back to a retrieval failure upstream, with the generator taking the blame.

The basic pipeline executes retrieval exactly once, in a fixed way, for every query. Each advanced technique in the next section relaxes one piece of that rigidity.



The **needle-in-a-haystack** test hides one fact in filler text and asks for it back, sweeping the fact's position and the total length. Accuracy stays high for short inputs but sags at mid-depth once the context grows: the model reads the ends better than the middle.

Pasting whole corpora into the prompt is possible, yet retrieval survives:

- **Cost.** Prompt tokens are billed and recomputed per query: a 1M-token prompt runs seconds and costs about a dollar, versus a fraction of a cent at milliseconds for 2K retrieved tokens.
- **Attention quality.** As the heatmap shows, more context can lower accuracy, so filling the window is not free.
- **Freshness and access.** The corpus changes hourly and varies per user's permissions; an index handles both.

Where long context wins: reasoning over one bounded artifact, like summarizing *this* contract. When the task needs all of the input, selection has nothing to select.

The previous lecture ended with behavior; this table adds the knowledge column and completes the decision framework:

	Prompting	RAG	Fine-tuning
Changes weights	no	no	yes
Adds new knowledge	context-limited	yes, unbounded	poorly, and it is static
Changes behavior/format	somewhat	no	yes
Knowledge freshness	per-prompt	index update	retraining
Citations possible	manual	native	no
Serving cost	prompt tokens	retrieval + tokens	one-time training

- The methods compose: production assistants routinely run a **fine-tuned** model (tone, format, refusals) over **retrieved** context (facts) with a carefully **prompted** template.
- Rule of thumb: fine-tune for *how*, retrieve for *what*.



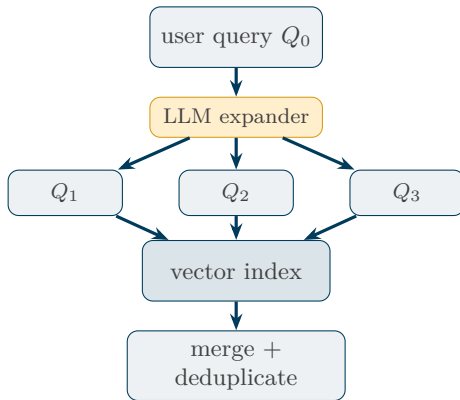
Table of Contents

5 Advanced RAG

- ▶ The Knowledge Problem
- ▶ Embeddings and Similarity Search
- ▶ Vector Databases
- ▶ The RAG Pipeline
- ▶ **Advanced RAG**
- ▶ Evaluating RAG
- ▶ Case Study: RAG on One Laptop

The user's raw text is often a poor search key, so advanced pipelines spend one cheap LLM call fixing it before retrieval:

- **Rewriting:** strip filler, resolve pronouns from chat history, normalize jargon.
- **Expansion (multi-query):** generate several paraphrases and sub-questions, retrieve for each in parallel, merge and deduplicate.
- **Decomposition:** split a compound question (*"compare X and Y"*) into retrievable parts.



Three parallel searches cover three phrasings of one need; their union beats any single embedding of Q_0 .

- **Query-document asymmetry:** queries are short questions, documents are long declarative prose. Even a good embedder places them in somewhat different regions, since it was trained mostly on document-shaped text.
- **Hypothetical Document Embeddings** (Gao et al., 2022): ask an LLM to *write a fake answer* to the query, embed that hypothetical document, and search with its vector instead of the query's.
- The fake answer is usually wrong in its facts and right in its *form*: it uses the vocabulary, tone, and structure of the documents being sought, so it lands nearer to the true passage than the raw question does.
- Costs one generation per query; helps most on short or underspecified queries, and can hurt when the LLM's hypothetical drifts off-domain.

The move to remember

Retrieval compares *like with like*. When query and corpus are shaped differently, transform the query until it is shaped like the thing it is looking for.

Enterprises rarely have one index. Knowledge is spread across vector stores, SQL databases, wikis, and APIs, and the right backend depends on the question:

- **A lightweight router** (a small LLM or classifier) reads the query and dispatches: semantic questions to the vector store, aggregations (*“how many orders last month”*) to SQL, entity lookups to the graph or CRM.
- Routing also selects *within* a store: which collection, which tenant, which date partition, saving both latency and irrelevant matches.
- The router is a natural place to decide **whether to retrieve at all**: a greeting or a pure-reasoning question needs no search, and skipping it saves the full retrieval latency.

This is the first appearance of a pattern that grows into a theme: an LLM making *control-flow decisions* inside the pipeline rather than just generating the final answer. A later lecture builds entire systems out of that idea.

- Run **dense and sparse retrieval in parallel** over the same corpus and merge the ranked lists. Each covers the other's blind spot: embeddings bridge paraphrase, BM25 nails exact identifiers.
- The merge has a problem to solve first: the two scores are **incomparable**. Cosine similarity lives in $[-1, 1]$; BM25 is unbounded and corpus-dependent. Normalizing scores is brittle because their distributions shift with every query.
- The standard solution ignores scores entirely and fuses **ranks**, which are always comparable. That is reciprocal rank fusion, next frame.

Why hybrid is the production default

A real query stream is a mixture: conversational paraphrases, half-remembered form numbers, drug names, error codes. No single retriever family covers that mixture, and the second retriever costs little since BM25 is nearly free.

Each ranked list $r \in R$ contributes through the document's *rank*, with $k = 60$ by convention (Cormack et al., 2009):

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

- Rank 1 contributes $\frac{1}{61} \approx 0.0164$, rank 10 contributes $\frac{1}{70} \approx 0.0143$: top ranks matter more, smoothly.
- k dampens the head: without it, rank 1 would dominate rank 2 by $2\times$.
- A document missing from a list simply contributes nothing from it.

Worked example ($k = 60$, two lists):

Chunk	Dense	BM25	RRF score
A	1	3	$\frac{1}{61} + \frac{1}{63} = .0323$
B	2	1	$\frac{1}{62} + \frac{1}{61} = .0325$
C	3	—	$\frac{1}{63} = .0159$
D	—	2	$\frac{1}{62} = .0161$

B edges out A by appearing near the top of *both* lists; C and D, each championed by only one retriever, fall behind. Agreement between retrievers is the strongest relevance signal available without a model.

Two-Stage Retrieval: Bi-Encoder, Then Cross-Encoder

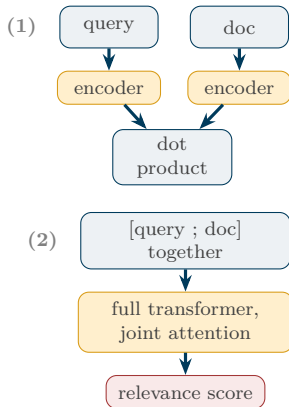
5 Advanced RAG

Bi-encoder (stage 1):

- Query and document encoded **separately**; the interaction is a single dot product, and document vectors are precomputed so search stays fast.
- Price: no query token ever attends to a document token.

Cross-encoder (stage 2):

- Reads [query; doc] **jointly** through full attention, so every term aligns with every term, resolving negation and conditions; emits one score.
- Price: one transformer pass *per pair*, impossible to precompute.

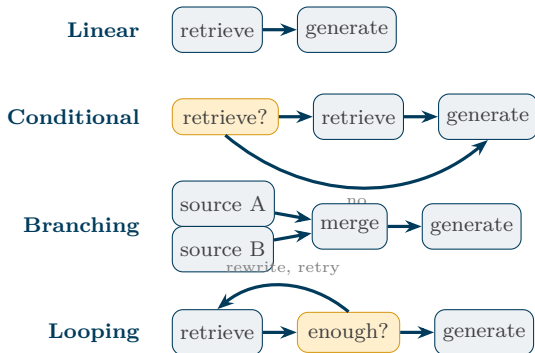




- Stage 1 is tuned for **recall**: cast a wide net cheaply and tolerate junk. Stage 2 is tuned for **precision**: spend model capacity only on the survivors.
- The budget math: reranking 100 candidates with a small cross-encoder costs about as much as generating a sentence; reranking the whole corpus would take hours.
- The same funnel shape appears across ML systems (ad ranking, recommendation, speech): cheap-and-wide feeds expensive-and-narrow.

Between reranking and the prompt, cheap set-level operations buy quality and tokens:

- **Deduplication:** overlapping chunks and multi-query expansion produce near-duplicates; embedding-similarity thresholds catch them before they waste context slots on the same paragraph twice.
- **Diversity:** if all five slots come from one document, a maximal-marginal-relevance pass swaps some for relevant chunks from other sources, hedging against a single wrong document.
- **Contextual compression:** a 500-token chunk may contain one load-bearing sentence. An extractive pass (or a small LLM) trims each chunk to its query-relevant core, cutting prompt cost and the distraction surface.
- **Ordering:** given uneven attention over long prompts, place the strongest evidence first or last rather than burying it in the middle.



- Systems evolve down this list as query diversity grows; each step adds a **decision** the pipeline previously hardcoded.
- **Conditional** saves latency on queries that need no search.
- **Branching** spans multiple corpora and merges (often with RRF).
- **Looping** inspects results and retries with a rewritten query, bounded by a retry budget.

Gold diamonds mark LLM decisions inside the control flow.



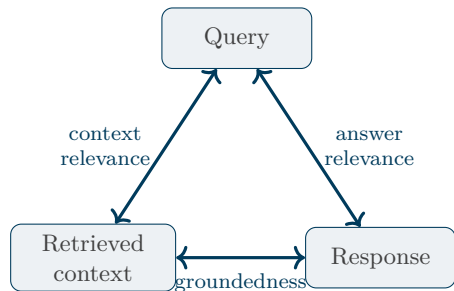
Table of Contents

6 Evaluating RAG

- ▶ The Knowledge Problem
- ▶ Embeddings and Similarity Search
- ▶ Vector Databases
- ▶ The RAG Pipeline
- ▶ Advanced RAG
- ▶ **Evaluating RAG**
- ▶ Case Study: RAG on One Laptop

- **BLEU** and **ROUGE** score n-gram overlap with a reference text. They served machine translation and summarization when outputs were short and references tight.
- A RAG answer can be **perfectly correct with near-zero overlap**: *“Reclean the room before the next admission”* vs. the reference *“the space must be recleaned prior to admitting another patient.”* Synonyms and word order destroy the score while preserving the meaning.
- The reverse failure is worse: an answer can copy 90% of the reference’s n-grams and negate the key clause. High BLEU, wrong answer.
- Correctness here is a question of **semantic equivalence against retrieved evidence**, which n-gram counting cannot represent.

So RAG evaluation splits into two measurable halves: **did we retrieve the right chunks** (ranking metrics, fully automatic) and **did we generate a faithful, relevant answer** (semantic judgment).



Three edges, three independent failure modes:

- **Context relevance:** did retrieval return chunks that address the query? (Grades the retriever alone.)
- **Groundedness:** is every claim in the response supported by the context? (Catches hallucination.)
- **Answer relevance:** does the response answer what was asked? (Grounded evasions fail here.)

Why grade the edges separately

A strong generator masks a weak retriever by answering from parametric memory. The composite score looks fine until a query arrives whose answer the model does not know, and the failure then surfaces in production.

Ground truth: for each test query, the set of chunks that contain the evidence.

Recall@ k : coverage of the relevant set within the top k :

$$\text{Recall@}k = \frac{|\text{relevant} \cap \text{top-}k|}{|\text{relevant}|}$$

Answers: *if the generator reads k chunks, is the evidence in there?* Choose k to match the context budget.

Worked example. Relevant chunks for a query: $\{c_7, c_{12}\}$. System returns $(c_3, c_7, c_9, c_{12}, c_1)$. Reciprocal rank = $\frac{1}{2}$ (first hit at rank 2); Recall@3 = $\frac{1}{2}$ (only c_7 in top 3); Recall@5 = 1.0.

Mean reciprocal rank: position of the *first* relevant hit, averaged over queries:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Rank 1 scores 1.0, rank 2 scores 0.5, rank 4 scores 0.25: steep, so MRR rewards putting evidence *on top*.

Discounted cumulative gain sums relevance discounted by rank, normalized by the ideal ordering:

$$\text{DCG}@k = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)}, \quad \text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}$$

- rel_i : relevance of the chunk at rank i (binary here; graded in general).
- IDCG is the DCG of the *ideal* ordering, so $\text{NDCG} \in [0, 1]$, comparable across queries.
- MRR sees only the first hit; NDCG credits **every** relevant chunk and its position.

Worked example ($k = 3$, relevant = $\{c_7, c_{12}\}$, returned (c_3, c_7, c_{12})). Only ranks 2 and 3 are hits:

$$\text{DCG}@3 = \frac{0}{\log_2 2} + \frac{1}{\log_2 3} + \frac{1}{\log_2 4} = 1.131$$

Ideal ordering puts both hits first:

$$\text{IDCG}@3 = \frac{1}{\log_2 2} + \frac{1}{\log_2 3} = 1.631, \text{ so}$$

$\text{NDCG}@3 = \frac{1.131}{1.631} = 0.693$. The metric charges the system for the irrelevant c_3 occupying rank 1.

Grading Generation: Groundedness and Answer Relevance

6 Evaluating RAG

- **Groundedness (faithfulness).** Decompose the response into atomic claims; check each against the retrieved context, using an NLI model or an LLM judge. The score is the fraction of supported claims. Unsupported claims are exactly the hallucinations that grounding was supposed to prevent.
- **Answer relevance.** A response can be fully grounded and still useless: “*the policy discusses cleaning verification*” is supported by the context and answers nothing. Judged by comparing the response against the *question*, independently of the context.
- With a labeled test set (as in our case study), answer **correctness** against a ground-truth reference replaces or complements these reference-free checks.
- All three generation-side scores require semantic judgment, which in practice means a model grading a model: the subject of the next frame.

- The previous lecture met this idea as RLAIIF, where model feedback replaces human preference labels during training. Evaluation is the same move at test time: a **judge model** grades outputs against a rubric.
- The judge prompt is engineered for **reproducibility**: a fixed rubric with anchored scale points (*5 = fully correct ... 1 = completely wrong*), the question, the reference or context, the candidate answer, and an output format of exactly one integer, decoded greedily.
- Judges scale where humans cannot: thousands of (query, answer) pairs per hour, re-run on every commit, cheap enough to sit in CI as a regression gate.
- Known biases carry over directly, and two are RAG-specific: judges favor **longer** answers (verbosity bias) and favor answers **stylistically similar to their own family's** output (self-preference), so the judge should come from a different family than the generator (Zheng et al., 2023).

Mitigations that work:

- Different model family for the judge, or two judges cross-checking.
- Swap candidate order across runs to cancel position bias in pairwise setups.
- Anchor the rubric with worked exemplar answers per score.
- Calibrate: score a sample with humans, measure judge–human agreement before trusting the judge at scale.

The stack in practice:

- **Golden dataset:** a few hundred curated (query, evidence, answer) triples; every pipeline change is scored against it offline.
- **RAGAS** (Es et al., 2023) and similar frameworks automate the triad’s metrics.
- **Online telemetry:** thumbs up/down, regenerate-rate, and abandonment complete the picture after deployment, since golden sets drift from real traffic.



Table of Contents

7 Case Study: RAG on One Laptop

- ▶ The Knowledge Problem
- ▶ Embeddings and Similarity Search
- ▶ Vector Databases
- ▶ The RAG Pipeline
- ▶ Advanced RAG
- ▶ Evaluating RAG
- ▶ Case Study: RAG on One Laptop

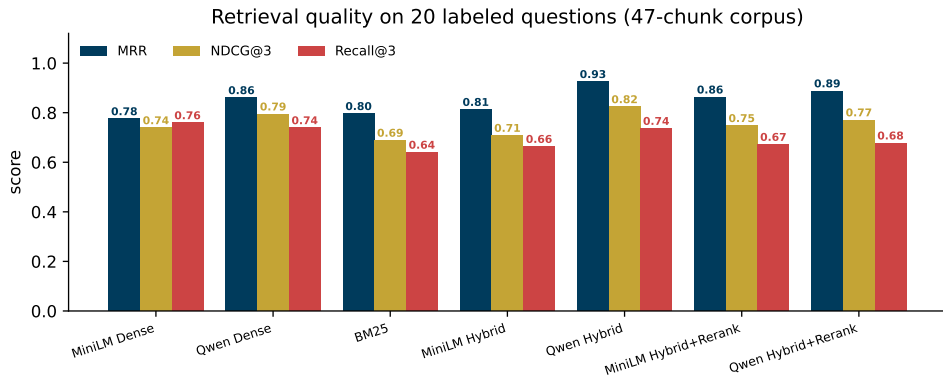
Corpus: four fictional hospital-policy PDFs (infection control, patient safety, medication administration, EHR standards); 47 page-level chunks.

Evaluation set: 20 hand-labeled questions, each with a ground-truth answer and the exact source pages, so every ranking metric from the last section is computable.

Strategies: dense, hybrid (RRF), and hybrid + rerank, each with both embedders, plus BM25: seven in total.

Role	Model	Size
Embedder A	all-MiniLM-L6-v2	22M
Embedder B	Qwen3-Embedding-0.6B	0.6B
Reranker	ms-marco-MiniLM-L-6	22M
Generator	Qwen3-1.7B (4-bit)	1.7B
Judge	Gemma-3-1B-it (4-bit)	1B

The generator is the same model the previous lecture fine-tuned; today its weights stay frozen and the knowledge arrives through the prompt. The judge comes from a different family, applying the self-preference lesson directly. At 47 chunks, dense search is an exact NumPy matrix multiply; no ANN index needed.



Qwen Hybrid leads MRR at **0.925**; the 22M-parameter MiniLM Dense, 25× smaller than its rival, posts the best Recall@3 of the entire table at **0.759**.

- **The small embedder holds its ground.** MiniLM Dense trails Qwen Dense by 0.085 MRR yet wins Recall@3 (0.759 vs 0.739). On 47 well-separated chunks there is little ambiguity for the extra 0.58B parameters to resolve; embedder capacity pays off on large, messy, multi-domain corpora.
- **Hybrid helps the strong embedder most.** Qwen Dense $\rightarrow$ Qwen Hybrid gains +0.065 MRR (0.860 $\rightarrow$ 0.925): BM25 catches the exact-string questions (thresholds, form names) that embeddings blur.
- **Reranking cuts both ways.** It lifts MiniLM Hybrid (0.812 $\rightarrow$ 0.863) and *lowers* Qwen Hybrid (0.925 $\rightarrow$ 0.887). A 22M reranker trained on web-search pairs can overrule a ranking that was already better than its own judgment. Stage 2 must be stronger than stage 1, or it subtracts value.
- **BM25 alone reaches 0.796 MRR** with no GPU, so the baseline stays cheap to beat.

The Same Question, With and Without Retrieval

7 Case Study: RAG on One Laptop

Query: *What is the maximum acceptable reading on an ATP bioluminescence test for a cleaned surface, and what happens if a room fails this verification?* (The threshold exists only in these fictional policies.)

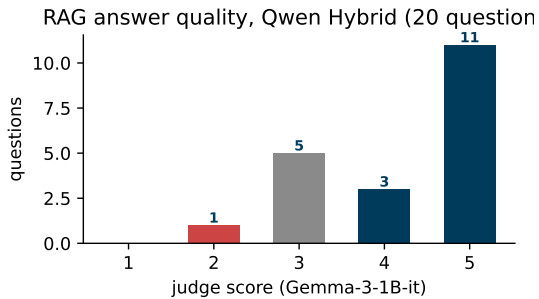
Without retrieval

The generator invents a number: “the maximum acceptable reading is typically **below 100 ATP units.**” It reads as authoritative, yet nothing in its weights holds this fictional policy, so the figure is wrong.

With retrieval (Qwen Hybrid)

“A cleaned surface must read **less than 250 relative light units (RLU)**. A room that fails must be recleaned before the next patient admission.” Grounded in `hospital_infection_control_policy.pdf`, p. 9.

The weights did not change between these two answers. Only the prompt did.



Gemma-3-1B scored all 20 generated answers on a 1–5 groundedness scale.

- Mean score **4.20** / 5
- **70%** rated 4 or 5 (grounded and complete)
- Distribution: {5:11, 4:3, 3:5, 2:1}

The low scores are the honest ones: multi-part questions where the answer covered only part of the ground truth. A same-family judge would have scored the generator higher, so the cross-family choice is what makes 4.20 trustworthy.

The full study, ingest to judged with all five models resident at once, ran in **0.5 minutes** at **2.64 GB** peak GPU memory on one laptop; an earlier 9B/7B version took 43 minutes for the same conclusions.

Everything so far assumed the pipeline *always* retrieves, once, before generating. The next step relaxes that assumption.

- **Retrieve conditionally.** A model can decide whether a query needs external evidence at all. “What is $2+2$?” should skip the index; “What does our 2026 refund policy say?” should not.
- **Retrieve iteratively.** A hard question can trigger several searches, each query informed by what the last one returned, until the model has gathered enough to answer.
- **Retrieval becomes a tool.** The index stops being a fixed pipeline stage and becomes one action the model chooses to invoke, alongside calculators, code execution, and web search.

That shift, from a fixed pipeline to a model that decides when and what to retrieve, is where agents begin. It is the subject of the next lecture.

1. **RAG separates knowledge from weights.** Fine-tuning teaches skills and style; retrieval supplies facts that change faster than any training run can keep up with.
2. **Dense and sparse retrieval fail differently.** Embeddings capture meaning; BM25 catches exact strings. Hybrid search with rank fusion recovers what either alone would miss.
3. **Vector databases trade recall for speed.** Exact search is $O(N)$; IVF, HNSW, and product quantization buy sub-linear latency at a measurable, tunable recall cost.
4. **Chunking and the five design choices decide quality** more than model size does. The corpus, chunk size, embedder, retrieval strategy, and prompt are the real knobs.
5. **Bigger is not automatically better.** A 22M embedder won Recall@3 here, and a reranker *hurt* the strongest pipeline. Measure on your own data.
6. **Evaluate retrieval and generation separately.** Recall@k and MRR grade the retriever; groundedness and answer relevance, scored by a cross-family judge, grade the generation.

Homework 10 is posted on Blackboard and is due **July 31**.

Companion notebooks. Two versions of today's case study ship with this lecture:

- `Lecture10_SEAS8520_Local_AppleMLX.ipynb` runs the full seven-strategy study on Apple Silicon in about half a minute. Start here: change the chunk size, swap an embedder, and watch the metrics move.
- `Lecture10_SEAS8520_Colab_CUDA.ipynb` runs the same pipeline on a free Colab T4 for anyone without a local GPU.

Looking ahead. The next lecture turns retrieval into one tool among many and hands the model the decision of when to use it: the move from pipelines to agents.